

Computer Networks — Transport Layer (Assignment 2)

Dual-Panel Activity & Transport-Layer Protocol Visualizer (L4 / L7 Extension)

Author / Student: Jitul Roy

Live Working Demo: <https://application-layer-six.vercel.app/>

AI Platform & Model: Google Antigravity • Gemini 3.8 Flash (High Reasoning)

Roll Number: 202402021026

GitHub Repository: <https://github.com/zitulroy/Application-layer>

Prerequisite: Working Assignment 1 (Application Layer Dashboard)

1. Summary of Changes Made in This Session

During this session, the existing Assignment 1 Application Layer dashboard was systematically upgraded into a dual-panel Transport & Application Layer protocol visualizer meeting all Assignment 2 requirements:

- Dual-View & Split-View Visualizer:** Extended the right panel to support three view modes toggled via an interactive tab bar: *Transport Layer (L4)*, *Application Layer (L7)*, and *Split View (L7 + L4)* which displays both layers simultaneously in lockstep.
- Complete RFC 793 / RFC 9293 Transport Simulation:** Built an accurate protocol state engine simulating TCP 3-way handshakes (SYN → SYN-ACK → ACK), data segments with flags ([PSH, ACK]), cumulative acknowledgments, and graceful 4-way connection teardowns ([FIN, ACK]).
- Interactive Packet Header & Logic Inspector:** Implemented a 32-bit TCP / 8-byte UDP header inspector box displaying RFC bitfield structures (Source/Destination Ports, Seq #, Ack #, Data Offset, Flags, Window Size, Checksum) and plain-English byte-accounting calculations.
- Real-Time Connection Telemetry:** Added a live telemetry bar displaying Client/Server TCP states (SYN_SENT, ESTABLISHED, FIN_WAIT_1, TIME_WAIT, CLOSED), Congestion Window (cwnd), and Receiver Window (rwnd).
- Streaming Protocol Comparison (Extra Credit):** Added an interactive toggle between **TCP (HLS / DASH)** segmented streaming and **UDP (Live RTP)** datagram delivery to demonstrate reliable byte-stream delivery versus low-latency, connectionless transmission.
- Automated Testing & Deep-Linking:** Added URL query parameter support (?act=...&step=...&view=...&proto=...) allowing direct linking and headless test capture. Verified layout with high-resolution screenshot captures in screenshots/.
- Submission Deliverables:** Generated reflection documentation in Markdown (REFLECTION.md), Word (reflection_document.docx), and print-ready PDF (reflection_document.pdf).

2. Dual-Panel Synchronization Architecture

The architecture links the left activity panel with the dual visualization tracks in the right panel through a single coordinated state engine:

- Hierarchical Data Binding:** Every L4 transport packet (TCP/UDP) stores an explicit pointer (l7Index) referencing its parent L7 transaction (DNS query, HTTP request/response, SMTP envelope command, or media segment).
- Unified Timeline Clock:** A master cursor coordinates both tracks. When stepping or auto-playing, packet *i* highlights on the L4 sequence diagram while simultaneously pulsing transaction `pkt[i].l7Index` on the L7 track.
- Comprehensive Controls:** Users can Play/Pause (Spacebar), Step Forward (> / Right Arrow), Step Backward (< / Left Arrow), Replay (↺), and adjust simulation speed (1x, 1.5x, 2x).

3. What the AI Got Wrong & How It Was Corrected

AI-assisted code generation initially produced subtle protocol bugs that required manual intervention and RFC verification:

Error Category	AI Initial Output	RFC Standard & Applied Correction
SYN / FIN Sequence Space	Treated SYN and FIN as consuming 0 sequence bytes because their payload Length = 0, causing HTTP GET to erroneously reuse Seq = 1000.	RFC 793 / 9293 Rule: SYN and FIN each consume exactly 1 byte of sequence space . Corrected sequence logic: <code>cSeq += 1</code> on SYN/FIN. HTTP GET correctly starts at Seq = 1001 and the peer returns Ack = 1001.

Error Category	AI Initial Output	RFC Standard & Applied Correction
Cumulative ACK Calculation	Set ACK numbers equal to the received segment's sequence number ($Ack = Seq$), confusing acknowledgment with echo.	RFC 9293 Standard: An ACK number specifies the <i>next expected byte</i> ($Ack = Seq + Length$). Corrected: 185-byte HTTP GET at Seq 1001 triggers Server $Ack = 1001 + 185 = 1186$. Pure ACKs ($Len = 0$) do not advance sequence space.
TCP Connection Teardown	Generated an abrupt 2-packet close (Client FIN → Server ACK), leaving half-open connections without a server FIN.	Full 4-Way Teardown: Modeled full bidirectional close: Client FIN, ACK (FIN_WAIT_1) → Server ACK (CLOSE_WAIT → FIN_WAIT_2) → Server FIN, ACK (LAST_ACK → TIME_WAIT) → Client Final ACK (CLOSED after 2MSL).
Streaming Protocol Modeling	Assumed video streaming must be strictly TCP HLS, omitting the datagram delivery comparison highlighted in the assignment prompt.	TCP vs. UDP Toggle: Added an interactive transport protocol selector enabling real-time comparison between TCP segmented byte-stream transfer and UDP stateless RTP datagram streaming.

4. Key Differences Observed Across Transport-Layer Flows

- Web Browsing (HTTP/1.1 over TCP):** Short-lived request-response lifecycle. Handshake (3 packets) and teardown (4 packets) account for ~50% of the exchange. Asymmetric payload (small ~185 B HTTP request vs multi-packet HTML response). Socket closes gracefully after page load.
- Mail Delivery (SMTP over TCP):** Conversational, lockstep protocol. The server speaks first with a 220 banner, followed by serialized command-response pairs (EHLO → 250, MAIL FROM → 250, RCPT TO → 250, DATA → 354, body → 250, QUIT → 221). Messages use [PSH, ACK] for immediate application delivery, making SMTP highly sensitive to latency and Round-Trip Time (RTT).
- Media Streaming (HLS over TCP vs. UDP RTP):** TCP streaming showcases *persistent connection reuse* (HTTP Keep-Alive), requesting media segments (seg000.ts, seg001.ts) over an established socket without repeated handshakes, while Congestion Window (cwnd) ramps up via Slow Start (1 → 2 → 4 → 8 → 16 MSS). In contrast, the UDP RTP option illustrates zero-RTT startup, stateless 8-byte header datagrams, and zero head-of-line blocking, prioritizing immediate playback over lossless delivery.

5. AI Platform & Model Selection Rationale

Google Antigravity with **Gemini 3.8 Flash (High Reasoning)** was chosen because it allows direct in-workspace editing and testing without external copy-pasting. Its strong protocol reasoning ensured RFC mathematical accuracy, while integrated command execution enabled automated screenshot generation and code verification.